

CRMProject — Estado y plan de inicio

Qué le falta al proyecto y por dónde empezar (orden recomendado)

Proyecto: CRMProject · Fecha: 31 de mayo de 2026 · Stack: Django + DRF + PostgreSQL + JWT

Parte 1 — ¿Qué le falta al proyecto?

Tienes un **núcleo de API CRM funcional**, pero aún estás lejos de un **SaaS B2B multi-empresa** listo para producción.

✓ YA TIENES

- Usuario custom (email, roles, auditoría)
- Dominio: Customer, CallLog
- API REST (usuarios, clientes, llamadas)
- Login JWT y IsAuthenticated por defecto
- Permisos por rol (admin, manager, vendedor por agente)
- Admin Django configurado
- PostgreSQL vía secrets.json

✗ FALTA (RESUMEN)

- Multi-tenancy (Organization)
- Auth completa (refresh, /me, rutas /api/)
- Tests, requirements.txt, README útil
- Producción, CORS, secretos fuera de git
- Paginación, OpenAPI, CRUD completo de llamadas

1. Multi-tenancy (salto a SaaS B2B)

- No hay modelo Organization ni FK en User / Customer.
- Email de cliente único *globalmente*; en B2B debe ser único *por empresa*.
- Querysets filtran por agente/rol, no por tenant.

2. Autenticación y API ordenada

Elemento	Estado
POST /auth/refresh/	No expuesto en URLs
GET /auth/me/	No existe
Prefijo /api/	Rutas en raíz (login/, customers/)
Registro / logout	No implementado

3. Cobertura API de ventas

- CallLog: solo listado y creación (sin detalle, edición ni borrado).
- Sin paginación, filtros ni búsqueda en DRF.
- Sin versionado de API.

4. Tests y calidad

- users/tests.py y sales/tests.py vacíos.
- Sin requirements.txt ni CI.

5. Infraestructura y producción

- Solo settings local / test; sin producción.
- secrets.json puede estar en git (riesgo de credenciales).
- open("secrets.json") depende del directorio de trabajo.
- Sin CORS, Docker ni logging estructurado.

6. Documentación y extras (más adelante)

- README genérico; sin OpenAPI/Swagger.
- Onboarding, invitaciones, facturación, auditoría avanzada.
- CallLog no usa AuditModel (inconsistencia menor).

Flujo: dónde estás vs. objetivo

Base actual (User + Sales + JWT + permisos por agente)



Auth JWT completo + rutas /api/

↓
Tests de permisos

↓
Organization / Multi-tenant

↓
Validaciones por org + onboarding + extras

Parte 2 — ¿Por dónde empezar?

Empieza **en orden**. Si saltas directo a multi-tenant con la API suelta y sin tests, vas a rehacer trabajo.

0

Hoy (30–60 min) — Proyecto reproducible y seguro Primero

1. Crear `requirements.txt` (Django, DRF, SimpleJWT, psycpg2, etc.).
2. Sacar `secrets.json` del repo: `.gitignore` + `secrets.json.example`.
3. Leer secretos con ruta basada en `BASE_DIR`, no path relativo al `cwd`.
4. README mínimo: `venv`, instalar, migrar, servidor.

1

Esta semana (1–2 días) — Cerrar auth y ordenar API

Tarea	Motivo
Prefijo <code>/api/v1/</code>	Separar admin, auth y recursos
<code>POST /api/auth/refresh/</code>	SimpleJWT ya lo trae; enlazar URL
<code>GET /api/auth/me/</code>	Perfil del usuario autenticado
Mover <code>login/</code> bajo <code>auth/</code>	Coherencia de rutas
Probar con Postman	Login → access → <code>/me</code> → <code>customers/</code>

Listo cuando: con JWT ves perfil y clientes; sin token, 401 (excepto login).

2

Siguiente sprint (2–4 días) — Primeros tests

5–8 tests de integración sugeridos:

- Login OK / credenciales incorrectas
- /me autenticado vs anónimo
- Vendedor solo ve sus clientes
- Manager/admin ve todos

3

Después — Multi-tenant (SaaS B2B)

Solo cuando pasos 1 y 2 estén estables.

1. Modelo Organization
2. User.organization y Customer.organization (FK obligatorias)
3. Unicidad: email de cliente por (organization, email)
4. Todo get_queryset() filtra por request.user.organization
5. Migración de datos existentes en local

Crear Organization, enlazarla a User y Customer, y filtrar por organización es lo que transforma el proyecto en CRM multi-empresa.

Qué NO hacer todavía

- Stripe, invitaciones, OpenAPI completo, Docker avanzado
- Validaciones finas “por organización” (después del paso 3)
- CRUD completo de CallLog (útil, pero secundario)

Primera tarea concreta (checklist)

1. requirements.txt + README corto
2. secrets.json fuera de git + secrets.json.example
3. urls: api/v1/auth/login/, refresh/, me/
4. Prueba manual: login → me → customers con Bearer token

Resumen en una frase

Empieza por **infra mínima (deps + secretos) + auth completa bajo /api/**; luego **tests**; después **multi-tenant**.

Si el objetivo es curso/portfolio (no SaaS)

Misma secuencia hasta el paso 2; **omite el paso 3** y añade paginación + Swagger + endpoints que falten en llamadas.